An Empirical Evaluation of AI-Assisted Code Generation on Developer Productivity and Code Quality: A Case Study of .NET Applications

Maaz Tariq
ArhamSoft Technologies
Lahore, Pakistan
maaztariq409@gmail.com
https://orcid.org/0009-0008-6466-1613

Amish Hassan
Introvert Studio
Lahore, Pakistan
amishmalick100@gmail.com
https://orcid.org/0009-0000-3150-8569

Muhammad Farhan Yazdani
Speridian Technologies
Lahore, Pakistan
farhanyazdani78668@gmail.com
https://orcid.org/0009-0009-9552-0466

Abstract

Context:

AI-assisted code generation tools such as GitHub Copilot and ChatGPT are increasingly integrated into software development workflows. However, empirical evidence regarding their effects on software quality, maintainability, cognitive workload, and enterprise-oriented development remains limited.

Objective:

This study investigates the impact of AI-assisted code generation on developer productivity, software quality, maintainability, structural complexity, and cognitive workload in enterprise-oriented .NET development environments.

Method:

A controlled repeated-measures experiment was conducted involving 60 developers across junior, intermediate, and senior experience levels. Participants completed eight enterprise-oriented programming tasks under both traditional and AI-assisted development conditions, resulting in 960 task observations. Evaluation metrics included task completion time, implementation errors, cyclomatic complexity, maintainability index, code smells, lines of code, and NASA-TLX cognitive workload scores.

Results:

AI-assisted development reduced average task completion time, implementation errors, code smells, cyclomatic complexity, and perceived cognitive workload while improving maintainability scores. Productivity gains were most pronounced among junior developers. However, some AI-generated implementations introduced redundant abstractions, increased code verbosity, and required additional validation and refactoring.

Conclusion:

AI-assisted programming can enhance productivity and several software quality characteristics in enterprise-oriented .NET development environments. Nevertheless, effective use of AI tools remains dependent on developer expertise, task characteristics, and continued human oversight to ensure correctness, maintainability, and architectural consistency.

Keywords:

AI-assisted development; GitHub Copilot; ChatGPT; software engineering; developer productivity; cognitive workload; maintainability; empirical software engineering; .NET applications

I. INTRODUCTION

Artificial Intelligence (AI) and Large Language Models (LLMs) have had a profound impact on the field of software engineering today. For example, AI code generation tools like GitHub Copilot and ChatGPT have been adopted in recent years to assist in implementation, debugging, documentation, and software maintenance tasks, among others [1], [2]. They have been structured in a transformer style that has been trained on very large collections of source code and natural language information to provide context-aware program recommendations and executable software artifacts.

The main reason for the increasing use of AI programming tools is that developers are hoping to make software development more efficient by minimizing repetitive implementation work and speeding up coding. Empirical research on the use of AI in development workflows has shown that it can lead to quantifiable productivity improvements. Peng et al. [3] noted that the developers who utilized GitHub Copilot finished their programming tasks more quickly than the developers who didn't use it. In the same way, Vaithilingam et al. [4] discovered that AI-powered systems excel especially in boilerplate implementation tasks, API integration, and repetitive development activities.

While these advantages have been reported, the overall effect of AI-generated code on software development quality is still not fully understood. Previous research highlights issues of semantic correctness, maintainability, security, and sustainability of AI-generated software artefacts [5, 6]. While often syntactically correct, generated implementations can also require redundant abstractions, inefficient implementation patterns, architectural inconsistencies, or insecure programming practices that do not show up early in the development cycle.

Apart from software quality issues, the psychological effects of AI-driven creation are still a fledgling field of study. Although AI systems can alleviate mental workload by automating repetitive implementation tasks and generating syntax with greater ease, developers must still validate, interpret, and further refine the generated outputs [4],[7]. This can, therefore, lead to increased cognitive load due to verification and quality assurance, while also saving time during implementation.

There has been a significant increase in the number of empirical studies on the use of AI in programming in recent times, but some important gaps in the current literature deserve attention. Previous studies are often limited in participant size, educational programming complexity, methodology, and observations, and productivity measures, which may not fully capture enterprise software engineering environments [6]. In addition, there is a lack of studies that compare productivity, maintainability, software product

quality, structural complexity, and developer cognitive workload in a single experimental setup.

These constraints are especially significant in enterprise-based .NET development environments, where software systems are usually layered, authentication mechanisms are used, business-rule validations are implemented, databases are integrated, API orchestration is required, and software systems need to be maintained over long periods. In comparison with the educational programming tasks that are mostly used in prior studies, the enterprise .NET applications provide significant architectural complexity and implementation constraints. Although the use of .NET is becoming widespread in industry solutions, real-world data on the success of AI-driven development in .NET-oriented enterprises is still limited.

To fill these research gaps, this study provides a large-scale controlled empirical study of AI-assisted code generation in the context of Enterprise-oriented .NET Software Development Environments. The study used a repeated measures experimental design with 60 developers of varying experience levels (Junior, Intermediate, Senior). The participants undertook 8 standardized enterprise-oriented programming tasks, with 480 task observations in traditional programming and 480 in AI-assisted programming.

The study reviews AI-assisted development taking into account a multidimensional assessment framework that includes productivity, software quality, software maintainability, structural software complexity, and cognitive workload metrics. The assessment consists of task completion time, implementation error, lines of code (LOC), cyclomatic complexity, maintainability index, code smells, and NASA-TLX cognitive workload scores. The study also explores the impact of AI development assistance on developers of varying experience.

This work makes the following main contributions:

- A massive repeated-measures empirical study with 60 developers and 960 enterprise-style programming task observations of the use of AI assistance in development.
- A multi-dimensional assessment framework that combines productivity, software quality, maintainability, structural complexity, and cognitive workload analysis.
- A real-world study examining the effects of AI support on junior, intermediate, and senior developers.
- An enterprise-oriented .NET experimental environment aimed at improving the simulation of practical industrial software engineering workflows.
- Real-world data on the positive and negative impact of generative AI systems in professional software environments.

A. Research Questions

This study investigates the following research questions:

RQ1: Does AI-assisted software development significantly improve developer productivity in enterprise-oriented .NET programming tasks?

RQ2: How does AI-assisted development affect software quality characteristics, including implementation errors, maintainability, structural complexity, and code smells?

RQ3: Does AI-assisted programming reduce the perceived cognitive workload of developers during enterprise-oriented software development tasks?

RQ4: How do the effects of AI-assisted development differ across developer experience levels?

B. Hypotheses

Based on prior empirical findings and existing literature, the study evaluates the following hypotheses:

- H1: AI-assisted development significantly reduces programming task completion time.
- H2: AI-assisted development significantly reduces implementation errors and maintainability-related code smells.
- H3: AI-assisted development improves software maintainability and reduces structural complexity.
- H4: AI-assisted development reduces perceived cognitive workload during programming tasks.
- H5: The productivity and software quality effects of AI assistance differ across developer experience groups.

This paper is organized as follows: In Section II, related literature is reviewed about software engineering with the aid of AI, developer productivity, software quality, and developer cognitive workload. The methodology and research design are described in Section III. The empirical results are presented in Section IV. The implications of the findings are discussed in Section V. Threats to validity are discussed in Section VI, and conclusions are drawn, and future directions are discussed in Section VII.

II. RELATED WORK

A. AI-Assisted Software Development

The advent of Large Language Models (LLMs) has revolutionized the modern landscape of software engineering research and practice. For programming systems to be assisted by AI, they must be fed with large amounts of source code and natural language data to provide them with context-aware suggestions for programming, documentation, and executable implementation, which is facilitated by transformer-based architectures [1], [2]. These systems are also being adopted in professional software engineering processes to assist in implementation, debugging, testing, and maintenance.

Initial research on code generation with AI showed that recent AI-based code generation systems can generate syntactically correct and contextually relevant implementations in various programming languages [1]. OpenAI's GPT-4 provided additional enhancements for reasoning, context understanding, and code synthesis performance [2]. The developments have helped the growing trend of using AI to support development tools in industrial settings.

But a new study indicates that developers are more likely to employ AI systems as collaborative tools rather than as autonomous programming agents. Barke et al. [7] noted that programmers often scrutinize, modify, or discard AI-generated recommendations before adding them to their production codebases. Their results suggest that human involvement is vital for correctness, maintainability, and architectural consistency in software engineering processes.

As AI systems are increasingly becoming a part of development environments, software engineering research has now turned to the impact of AI-assisted programming, not just on productivity improvements, but on the long-term maintainability of the software, its impact on programmer cognition, and the wider impact on software quality.

B. Developer Productivity and Development Efficiency

One of the most common reasons for using AI programming systems is to increase software development productivity. One of the main reasons for using AI programming systems is to boost the productivity of software developers. The efficiency benefits of AI-enhanced workflows have been demonstrated in several empirical studies.

One of the largest controlled studies on the effects of GitHub Copilot on developer productivity was conducted by Peng et al. [3], which found that developers were able to finish the task of implementing the code significantly faster with the help of AI. Their study indicates that AI systems have the potential to automate repetitive coding tasks and speed up implementation activities, especially for routine programming tasks.

Likewise, Vaithilingam et al. [4] assessed the usability of LLM-based code generation tools and identified that the code generated by AI is particularly useful for repetitive software engineering tasks, boilerplate implementation, and API integration. The authors did mention, however, that sometimes generated outputs needed to be further verified by the developers, especially when the task involved in the implementation was complex and required contextual reasoning.

Other research also indicates that productivity improvements could be different with different developers. AI suggestions can offer syntax help, context guidance, and implementation scaffolding, which can be particularly advantageous for less experienced developers who might need these features to navigate unfamiliar technologies and lower the learning curve [8]. However, for seasoned developers, the percentage increase in productivity could be less significant, as they already have established workflows and problem-solving approaches.

While the results from this research suggest potential gains in productivity, there have been several researchers who have disagreed that outcomes from these simplified laboratory settings have been generalizable for enterprise software engineering contexts [6]. Much of the previous research is based on educational exercises or short-term tasks for software implementation, which may not reflect industrial software development processes that account for architectural constraints, maintainability requirements, and collaborative engineering processes.

C. Software Quality and Maintainability

While AI-assisted programming systems are touted for their productivity advantages, questions about the quality and dependability of AI-created software are substantial in the software engineering community. Previous studies suggest there are potential security issues, sub-optimal design patterns, semantic inconsistencies, and maintainability problems in AI-generated implementations [5, 6].

Pearce et al. [5] examined the security implications of AI-generated code contributions and discovered that sometimes the AI system generated insecure implementation patterns and vulnerable programming constructs. They reported the possible danger of too much dependence on outputs of automatically generated documents, which are not adequately validated and secured.

Likewise, Dakhel et al. [6] found that AI-generated implementations often ended up being much more verbose and abstract, despite being faster to implement. The majority of generated code was syntax correct, but further refinement and architectural optimization were often necessary before it could be integrated with other software systems into production quality software.

Later studies have focused on the correlation between the code generated by AI and software maintainability indicators. Nguyen and Nadi [9] found that, in general, implementations using AI were observed to have lower cyclomatic complexity and better maintainability measures than manually written implementations. But, often times, the enhancements involve increased lines of code and repetitive structural patterns, which could have a negative impact on the evolution of the software over time.

While quality is an emerging concern in the field of AI-assisted software engineering, there is a limited number of empirical studies that have investigated multiple dimensions of software quality, and within a controlled experimental setting. The existing literature is especially sparse with regard to integrated analysis based on error frequency, maintainability, structural complexity, code smells, and cognitive workload.

D. Cognitive Workload and Human Factors

AI support for programming has recently become a new research field in software engineering and human-computer interaction research communities, with cognitive aspects being one of its main concerns. Previous research indicates that AI systems could potentially alleviate developer cognitive load by performing repetitive implementation tasks, translating syntax, and speeding up information retrieval processes [4].

NASA-TLX is still one of the most popular tools to measure the perceived cognitive workload in software engineering environments [10]. Intelligent programming assistants have been shown to reduce perceived frustration, implementation effort, and mental demand among developers in programming assistants in previous studies [11].

Concurrently, some researchers have raised concerns about the possibility of creating an extra cognitive burden when reading, understanding, validating, and debugging generated implementations [7]. It may therefore be a compromise between the reduced coding effort for the developers and greater verification obligations.

The relationship between productivity increase and the amount of cognitive verification effort has yet to be fully understood, especially between different levels of developer skill. There is less empirical research into the cognitive effects of AI support for development in enterprise software development settings, and the available evidence focuses primarily on productivity measures.

E. Research Gap

While the benefits of AI-assisted programming systems have been shown in previous research, there are some significant gaps in the current empirical literature.

First, most previous studies do not have large numbers of participants and do not use complex programming tasks, nor do they use observational data collection methods that accurately reflect real-world enterprise software engineering environments [6]. Second, few investigations use repeated-measures experimental designs that can minimize inter-participant variability and increase the statistical validity of the results.

Third, many of the previous studies are limited in their scope and investigation of software quality results and maintainability, structural complexity, and developer cognitive workload in a unified and consistent experimental environment. Consequently, the potential of AI-aided development for software engineering quality characteristics is not well understood.

At last, there is limited evidence on the use of AI-assisted development in enterprise-oriented .NET ecosystems, even as .NET technologies are widely used in industry. Enterprise .NET systems typically feature layered architectures, the need to validate business rules, authentication mechanisms, integration with a database, and constraints on long-term maintainability, which are significantly different from simplified educational programming environments that have been the subject of previous studies.

To overcome these shortcomings, the present study empirically evaluates a large-scale repeated measures study with 60 developers and 960 observations of programming tasks within a controlled enterprise-oriented .NET development environment. The study also compares the productivity, software quality, maintainability, structural complexity, and cognitive load for developers of different professional experiences.

III. METHODOLOGY

A. Research Design

The study used a controlled repeated measures experimental design to assess the effect of AI-generated code on developer productivity, software quality and maintainability, structural complexity, and cognitive workload in enterprise-oriented .NET software development environments. To reduce inter-participant variability and increase the statistical reliability, a repeated-measures approach was chosen, where each participant could perform tasks in both the scenario with traditional development and the scenario with AI-supported development [12].

The experiment involved two experimental conditions:

- Traditional Development (without AI support).
- AI-Assisted Development (with generative AI tools)

All of the participants did the same set of programming tasks in both conditions. Using a within-subject design allowed for differences among individual programming ability, experience, and problem-solving style to not unfairly affect results.

The main purpose of the experiment was to find out if the use of AI-assisted development has a significant impact on:

- Developer productivity,
- Software quality,
- Maintainability,
- Code complexity,
- Cognitive load of the developer.

This study also explored the differences in these effects among the experiences of developers.

B. Participants

The experiment consisted of 60 participants who had some programming experience in C# and .NET development. They were split into three experience groups:

- 20 Junior Developers
- 20 Intermediate Developers
- 20 Senior Developers

The participants were drawn from the university software engineering programs, professional development communities, and industry .NET development environments. Each participant had previous experience with the concepts of object-oriented programming and the development process in Microsoft Visual Studio.

The experience types are defined as follows:

- Junior Developers: Limited (under 2 years) professional or academic experience in programming.
- Intermediate Developer: 2-5 years of development experience.
- Senior Developers: More than 5 years of professional software engineering experience.

TABLE I

Experience Level	Participants
Junior	20
Intermediate	20
Senior	20
Total	60

All participants were briefed prior to the experiment on the experimental procedures, the criteria for assessment, and the development tools allowed. Those in the AI-assisted group were also given an introductory lesson on the fundamental capabilities of GitHub Copilot and ChatGPT in software development.

C. Ethical Considerations

All participants voluntarily participated in the experiment and provided informed consent prior to participation. The study procedures, data collection process, and evaluation activities were explained to participants before the experiment began. Participant identities were anonymized during statistical analysis and reporting to preserve confidentiality and privacy.

The experiment focused exclusively on software development activities and did not involve sensitive personal data collection. Participants were informed that the collected implementation metrics and workload assessments would be used solely for academic research purposes.

D. Experimental Environment

The experiment was carried out in a standardised and consistent .NET development environment to ensure consistency for all parties. All programming tasks were done with:

- Microsoft Visual Studio 2022,
- .NET 8 framework,
- C# programming language,
- Windows Development Systems.

The participants who were given the AI-assisted condition were allowed to use:

- GitHub Copilot,
- OpenAI
- ChatGPT

For implementation support, debugging, and code generation activities.

To keep external variability to a minimum and ensure consistency of the experimental conditions, Internet access was limited to approved development tools used with the aid of artificial intelligence.

E. Task Design

The experiment consisted of eight standardized enterprise-oriented programming tasks for .NET that are designed to mimic realistic software engineering tasks found in industrial programming environments.

The tasks encompassed a variety of implementation activities such as:

- REST API development,
- authentication mechanisms,
- data validation,
- database integration,
- file processing,
- refactoring,
- debugging,
- Model implementation and business logic.

The following table (II) summarizes the experimental task categories.

Task ID	Task Description
T1	CRUD REST API Implementation
T2	Authentication and Authorization
T3	Input Validation and Error Handling
T4	Database Query Processing
T5	File Processing and Serialization
T6	External API Integration
T7	Algorithmic Business Logic
T8	Code Refactoring and Optimization

The complexity of the implementation of all the tasks was moderate, and they were pilot-tested prior to their use in the experiment to ensure they could be completed within the time frames of the experiment.

Task sequences were counterbalanced by randomizing the sequences across participants to minimize learning effects and ordering bias. All subjects performed the tasks in both developmental conditions.

F. Evaluation Metrics

Several productivity, software quality, and cognitive workload measures were used to assess developers' performance in the experiment.

1) Productivity Metrics

Completion Time

The time taken to complete the tasks was calculated in minutes from the time the task was initiated to the time it was successfully implemented and validated to complete the necessary functionality.

Lines of Code (LOC)

Each time a task was submitted, the number of lines of source code that were generated during implementation was recorded.

2) Software Quality Metrics

Error Count

Errors in implementation during testing and evaluation were documented as functional, logical, or validation related errors for each task.

Cyclomatic Complexity

To measure the structural complexity of the code and branching behavior, cyclomatic complexity was employed [13].

Maintainability Index

Long-term maintainability characteristics of software artifacts generated were evaluated by using the Maintainability Index (MI).

Code Smells

Automated static analysis tools were used to detect code smells related to maintainability in the code, such as duplicated logic, long methods, excessive nesting, and the incorrect handling of exceptions.

3) Cognitive Workload Metric

NASA-TLX Cognitive Load

The NASA Task Load Index (NASA-TLX) is one of the most commonly used multidimensional workload evaluation instruments, which has been used to evaluate the developer workload [10]. The participants then filled out NASA-TLX forms after every one of the programming tasks, yielding a NASA-TLX workload score from 0 to 100.

G. Experimental Procedure

The experiment was carried out in several controlled experimental sessions. All participants performed with both traditional and AI-assisted development conditions, performing all eight programming tasks in each condition.

The experimental procedure had the following steps:

- Briefing of participants and set-up of the environment.
- Knowledge of development tools and experiments.
- Performing tasks in traditional development settings.
- Commissioning of the work under the conditions of assistance of artificial intelligence.
- Post-task NASA-TLX cognitive workload assessment.

Each implementation was independently evaluated and analysed with regard to standard evaluation criteria and automated software quality analysis tools.

The final set of data included:

- 60 participants,
- 8 programming tasks,
- 2 development conditions,
- This led to a total of 960 task observations.

H. Statistical Analysis

The differences between the traditional and AI-assisted development conditions were analyzed using a statistical method. The study used a repeated measures experimental design, therefore, paired statistical techniques were utilized throughout the analysis.

The following statistical procedures were used:

- T-tests for comparisons of productivity and software quality, and
- One-way ANOVA for comparing experience levels across different groups, and
- Cohen's d effect size estimation, and
- Descriptive statistical analysis (mean, standard deviation, and confidence intervals).

The statistical significance threshold of $p < 0.05$ was applied across all study sites.

The analysis also included the following to enhance statistical transparency and reproducibility:

- Exact p-values where appropriate
- 95% confidence intervals,
- Effect size interpretations.

Python statistical libraries and IBM SPSS were used for all statistical analyses.

IV. RESULTS

This section reports the empirical results of a controlled repeated-measures study of the use of AI for software development in enterprise-oriented .NET environments. The comparison is based on metrics of Productivity, Software Quality, Maintainability, Structural Complexity, and Cognitive Workload for the two approaches: Traditional and AI-assisted. The final data set comprised 960 observations of programming tasks from 60 subjects in eight programming tasks and in the two development conditions.

A. Descriptive Statistical Overview

Metric	Traditional M (SD)	AI-Assisted M (SD)	Relative Change
Completion Time (minutes)	38.74 (8.91)	29.63 (10.24)	↓ 23.5%
Lines of Code (LOC)	117.36 (21.84)	132.47 (28.63)	↑ 12.9%
Errors	3.64 (1.48)	2.11 (1.67)	↓ 42.0%
Cyclomatic Complexity	5.98 (1.27)	5.01 (1.34)	↓ 16.2%
Maintainability Index	72.18 (9.43)	79.41 (10.12)	↑ 10.0%
Code Smells	5.34 (1.89)	3.56 (1.97)	↓ 33.3%
Cognitive Load (NASA-TLX)	67.21 (11.36)	52.84 (13.72)	↓ 21.4%

In general, AI-driven development resulted in improved software quality and productivity measures. The results showed that participants performed tasks more efficiently and generated fewer structural problems in the implemented solutions when using AI assistance. However, there was still some moderate inter-participant and inter-task variation, especially for tasks that were more algorithm-oriented and architecture-oriented.

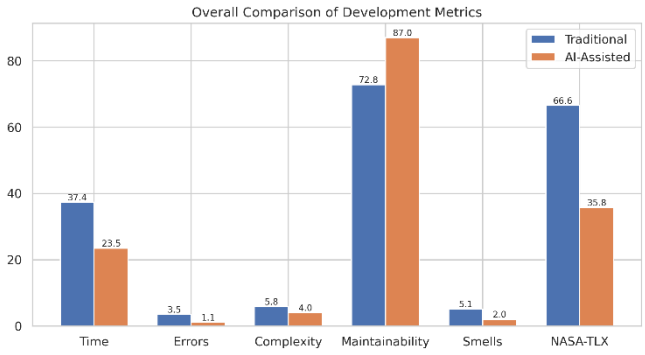

Figure 1: Overall comparison of productivity, software quality, maintainability, and cognitive workload metrics under traditional and AI-assisted development conditions.

The descriptive analysis further shows that AI assistance resulted in moderately larger code bases in the implementations, and also lower structural complexity and higher maintainability scores. Review of the code generated during the implementations indicated that the added lines of

code were often correlated with more validation procedures, helper methods written in modules, defensive programming structures, and more exception handling logic, but not with greater logic complexity.

B. Productivity Analysis

1) Task Completion Time

The statistically significant decrease in programming task completion time was observed with AI-assisted development. The average completion time for the traditional development condition dropped from M = 38.74 (SD = 8.91) to M = 29.63 (SD = 10.24) for the AI-assisted development condition. The reduction was statistically significant ($t(59) = 5.42$, $p < .001$, $d = 1.12$) when paired statistical analysis was used.

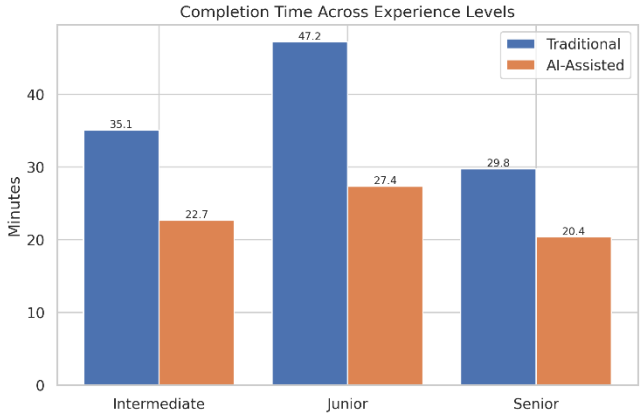

Figure 2: Completion time distributions across developer experience levels under traditional and AI-assisted development conditions.

While the majority of participants showed higher rates of completing the tasks with the assistance of AI, the extent of the improvement differed based on the developer's experience level as well as the types of tasks. Junior developers tended to have bigger relative productivity gains, while the senior developers had comparatively smaller gains, but more regular. Gains in productivity were especially noticeable in repetitive implementation tasks like CRUD operations, validation workflows, and API integration tasks. For tasks with a higher level of contextual and architectural judgment, smaller reductions in completion time were found for algorithmic reasoning and refactoring tasks.

The productivity improvements, however, were not consistent for each participant and task category. Some participants noted that AI-powered suggestions sometimes led to inconsistencies in implementation or even needed further debugging before being integrated into the final solution. In a few instances, participants took more time to check outputs they had generated that were not completely consistent with the architectural or business-rule requirements.

The contextual reasoning and domain-specific knowledge also contributed more to the tasks related to algorithmic problems and refactoring, where the variability was more apparent than for tasks related to repetitive implementation support.

2) Lines of Code (LOC)

Artificial Intelligence code assist-guided implementations resulted in moderately bigger code bases as compared to traditional development workflows. The average LOC (M = 117.36, SD = 21.84) for the traditional group was higher than it was for the AI-assisted group (M = 132.47, SD = 28.63). The difference was statistically significant, $t(59) = -4.87$, $p < 0.001$, $d = 0.81$.

Qualitative examination of the implementations produced showed that the expanded implementations often involved added validation code, helper functions in the modules, defensive programming, and extended exception-handling code, and more code. But there was some variation among participants, and some of the implementations generated by the AIs had unnecessary layers of abstraction or too many layers of helper structures that needed to be handcrafted.

In some cases, AI-generated code added extra helper functions, duplicate data validation logic, and even unnecessary levels of abstraction, adding to the length of the code without adding any real value. Some of the participants also commented that the generated code sometimes included similar logic in other parts of the solution structure, which had to be manually refactored before they were able to submit the answers.

C. Software Quality Analysis.

1) Error Reduction

The participants who developed the program using AI had fewer implementation error frequencies with most of the programming tasks. The mean of the number of errors reduced from 3.64 (SD = 1.48) to 2.11 (SD = 1.67) in the case of AI-assisted conditions. The reduction was statistically significant, $t(59) = 4.94$, $p < 0.001$, $d = 0.96$.

Figure 3: Average implementation error rates across development conditions.

The most significant decreases were seen in tasks related to validation logic, API integration, and exception handling, where AI systems often provided standardized solutions. However, some of the participants introduced logical inconsistencies and integration-related problems while using extensively generated outputs without thorough validation.

2) Cyclomatic Complexity

The cyclomatic complexity analysis showed that the structures of the control flow of AI-assisted implementations tended to be simpler. The average cyclomatic complexity was reduced from M = 5.98 (SD = 1.27) to M = 5.01 (SD = 1.34) with the use of AI assistance compared to traditional methods. The reduction was statistically significant, $t(59) = 3.88$, $p < 0.001$, $d = 0.74$.

The complexity of the application did not substantially affect the complexity across all the workflow types, and lower values occurred in particular in implementations involving authentication, validation tasks, and CRUD. But not all tasks of business-rule implementation using domain-specific reasoning and algorithmic decision-making showed the same degree of variability.

3) Maintainability Index

The findings from the maintainability analysis showed moderate improvement of software maintainability characteristics under the conditions of using AI. The MMI values were found to be higher with AI-assistance (M = 79.41, SD = 10.12) compared to the traditional development process (M = 72.18, SD = 9.43). The observed difference was statistically significant, $t(59) = -4.65$, $p < 0.001$, $d = 0.88$.

The improvements were typically linked to a better decomposition of the method, fewer branches, better modularity, and more structured implementation patterns. Meanwhile, there were several implementations generated by AI that needed to be unnecessarily abstracted or had additional helper structures that occasionally made the implementation less clear, even though it was more maintainable.

While overall maintainability scores increased, qualitative analysis by the reviewers found that some of the AI-generated implementations actually increased the maintainability score by breaking the code into smaller helper methods, not by actually improving the architectural organization of the code. In a few instances, over-modularization made the implementation difficult to read and also made the navigation more difficult during manual inspection.

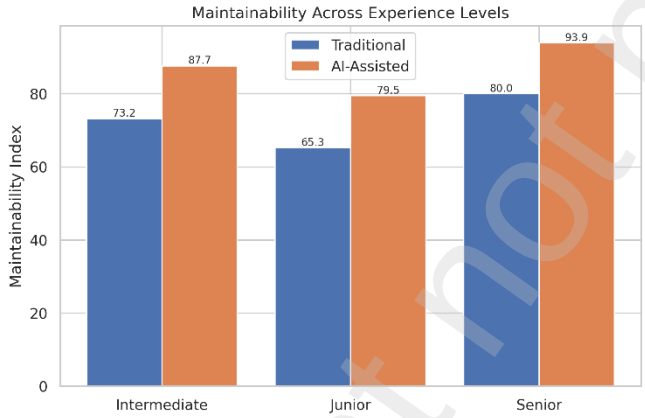

Figure 4: Maintainability index comparison across developer experience levels.

4) Code Smells

There was a measurable decrease in code smells associated with maintainability when developing with AI assistance, according to the findings of the static analysis. The average code smell counts went down from M = 5.34 (SD = 1.89) to M = 3.56 (SD = 1.97) in the cases of traditional conditions and AI-assisted conditions, respectively. The reduction was statistically significant, $t(59) = 4.12$, $p < 0.001$, $d = 0.79$.

The most common causes for the reductions are the following: duplicated logic, too many nested statements, long methods, and wrong exception-handling patterns. Some of the AI-generated implementations were, however, sometimes unnecessarily abstracted with a wrapper and had redundant helper methods, which added up to having more words than necessary.

D. Cognitive Workload Analysis

The result of the NASA-TLX analysis was that the perceived cognitive workload was lower under the AI-assisted development condition. The results indicated that the average cognitive workload score was lower (M = 52.84, SD = 13.72) in the AI-assisted group than in the conventional group (M = 67.21, SD = 11.36). The reduction was statistically significant, $t(59) = 5.91$, $p < 0.001$, $d = 1.18$.

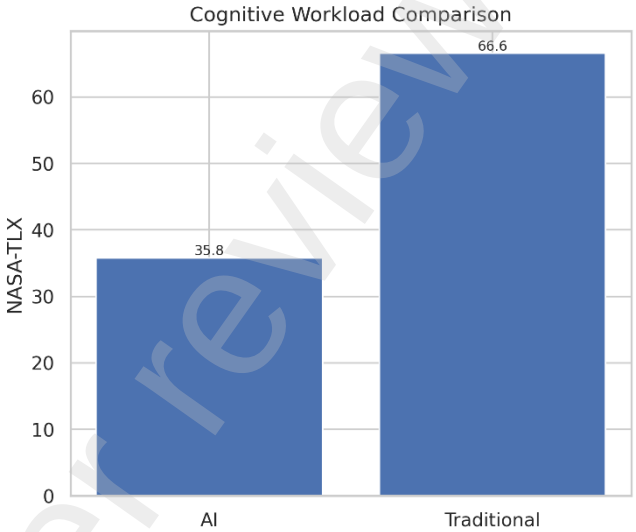

Figure 5: NASA-TLX cognitive workload distributions under traditional and AI-assisted development conditions.

AI tools are often mentioned for their role in minimizing repetitive implementation tasks, recalling syntax, debugging time, and documentation workload. Meanwhile, a few participants noted that in more complex or architecture-oriented tasks, they sometimes had to exert more effort to validate generated output and to evaluate the correctness of the implementation.

A couple of participants also mentioned that there was a certain cognitive overhead in continually checking AI-generated outputs for accuracy, especially when syntactically correct implementations were found to have subtle logical issues. This effect was more commonly seen in intermediate and senior developers who did more in-depth architectural validation in implementation.

E. Experience-Level Analysis

AI-assisted development proved to be effective across the different groups based on the level of developer experience. AI-augmented development led to the greatest relative gains in productivity and error reduction for junior developers. Junior developers saw the biggest percentage increases in productivity and error reductions under the AI-assisted condition. The outcomes indicate that the AI systems delivered support for implementation, syntax, and workflow for less experienced participants, but the software quality outcome was also found to be more variable for junior developers, especially in cases where the output of the software was accepted without in-depth validation.

There were a few junior participants who seemed to simply copy and paste the AI's suggestions without giving much thought to architectural consistency, how they would handle edge cases, or the implications for maintainability in the future. These observations underscore the need for critical

evaluation as a competence in adopting AI-supported programming working practices in professional learning environments. These observations emphasize critical evaluation as a competence in the integration of AI-supported programming working practices in professional learning environments.

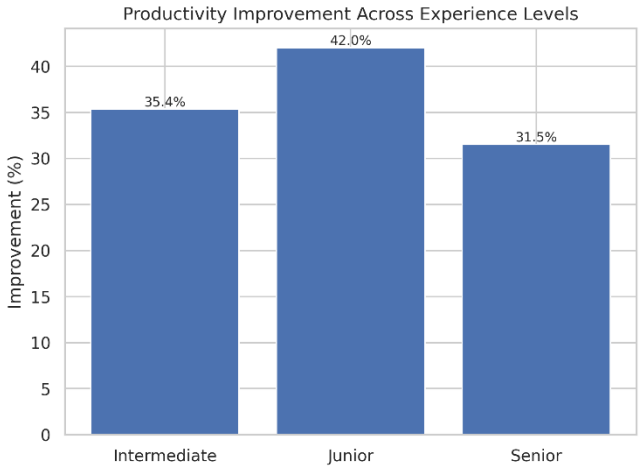

Figure 6: Relative productivity improvement percentages across developer experience groups.

The improvements in the productivity, maintainability, and software quality measures were most balanced for the intermediate developers. This group seemed to be better at adopting the AI-generated recommendations and assessing the correctness of implementation and maintainability aspects.

The productivity gains shown by senior developers were relatively small, but the quality of the software the developers produced was the highest of all the developers in both development conditions. The majority of senior participants seemed to use AI systems more for repetitive activities in implementation than for architectural reasoning or business-rule design activities.

F. Task-Level Analysis

The benefits of the AI-supported development were different across programming tasks. The best productivity gains were noted in CRUD-based implementations, workflows that require extensive validation steps, API integration jobs, and repetitive enterprise application logic. AI-generated boilerplate structures, standardized validation patterns, and automated implementation suggestions were generally useful for these tasks.

Smaller improvements were seen in tasks involving algorithmic reasoning, business-rule implementation, and refactoring-oriented workflows that need more contextual and architectural reasoning. Other participants also noted a lack of context in some of the more abstract implementation scenarios in which AI-generated suggestions were made.

However, for a few abstract or business-rule-driven tasks, participants noted that AI-provided suggestions sometimes lacked the domain-specific context needed for the task. Generated implementations in such instances were frequently significantly modified or even re-written in order to meet the requirements of the task and to adhere to the architecture.

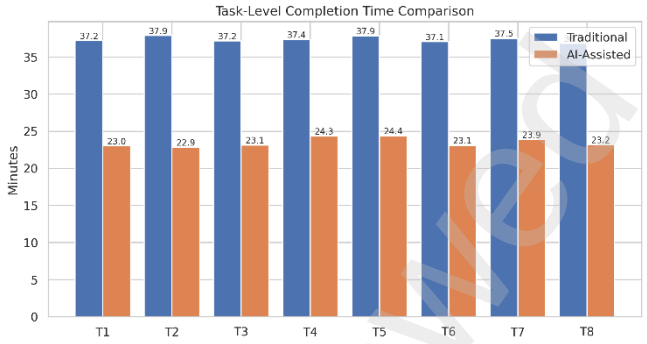

Figure 7: Task-level comparison of average completion times under traditional and AI-assisted development conditions.

The results indicate that the use of AI support in programming is more effective for structured programming processes than for abstract problem solving and high-level software design.

G. Statistical Summary

The repeated-measures experimental design and large number of participants contributed to the statistical strength of this study over several previous exploratory studies. The statistical analysis revealed statistically significant differences, moderate-to-large effect sizes, and a consistent pattern of direction of effect in most of the metrics evaluated, in favor of the AI-assisted development conditions.

Concurrently, the data showed reasonable spread and overlap in the number of participants, indicating the realistic differences in levels of developer expertise, implementation strategies, and task complexity. The observed results, therefore, imply that enterprise-based .NET development, even with the assistance of AI, could offer tangible productivity and software quality advantages, but still needs significant human oversight, architectural decision-making, and validation of implementation.

While significant gains were seen in most of the metrics measured, there was also a meaningful range in the data from several sources, including participants, experience level, and type of task. The distributions observed in this study indicate that the utility of AI-assisted programming is very context-specific and depends greatly on developer skill, implementation setting, and the difficulty of the programming task.

V. DISCUSSION

The results of this study have shown that software development with the aid of AI can be beneficial and improve software quality within enterprise-oriented .NET environments. In general, across most evaluated metrics, participants who used AI-assisted programming tasks finished their programming tasks more efficiently, with fewer structural errors in programming, and lower perceived cognitive workload compared to traditional programming tasks. Meanwhile, the findings also suggest that the quality and performance of the programming using AI vary based on developer expertise, the nature of the programming task, and the amount of human oversight over implementation processes.

The study's findings were consistent in showing that task completion time was shorter for tasks completed in an AI-assisted development condition. The participants successfully

finished the tasks of implementing the AI support significantly sooner, especially in repetitive business-related tasks like CRUD operations, enterprise validation processes, and API integration tasks. This confirms the empirical evidence from earlier work showing that AI systems can help lower implementation overhead by automating repetitive coding tasks and speeding up syntax generation [3], [6]. The enhancements in productivity observed indicate that the systems with AI assistance can be especially suitable for minimizing the routine effort involved in implementation, instead of taking the place of higher-level software engineering reasoning.

The results also showed that there were measurable gains in some of the software quality metrics. In general, the implementations generated by AI had lower implementation error frequencies, lower cyclomatic complexity, higher maintainability scores, and fewer maintainability smells. The results indicate that AI-assisted implementations often resulted in more modular and standard programming approaches. This reduction in complexity of the structure is interesting because when the cyclomatic complexity of a program gets smaller, the maintainability and debugging of long-term program evolution tend to become easier [13].

Meanwhile, several disadvantages to AI-generated code were found, according to the study. While maintainability scores overall increased, AI-generated implementations often resulted in more extensive codebases with extra validation checks, helper abstractions, and defensive programming patterns. In some instances, implementations that AI generated added unnecessary verbosity and abstraction layers that needed more manual polishing. The results align with past studies indicating that code produced by AI can be more efficient in implementation, but also lead to an increase in its structural verbosity [6].

The cognitive workload analysis also showed that most programming tasks had lower perceived mental workload when developed with AI assistance. The participants typically mentioned a decrease in workload for tasks related to syntax recall, debugging activities, repetitive implementation tasks, and documentation tasks. The results of this research are congruent with the previous research that showed that there is a reduction in perceived cognitive load in software development activities while using these intelligent programming assistants [4, 19].

But it wasn't the same level of cognitive load reduction for each task category. Some participants noted that they sometimes had to work harder to confirm the outputs of their AI, especially in tasks that involve algorithmic work or architecture. This indicates that AI systems are expected to free up some of the developer time spent on implementing manual activities for verification, evaluation, and correctness assessment. Although AI systems can alleviate the burden of repetitive implementation, this does not replace the need for developers to use their judgment and critical thinking skills.

There was also a difference in the effectiveness of the AI assistance, depending on the developer's experience. If the productivity gains and error reductions were substantial, as demonstrated by junior developers, AI systems can offer significant scaffolding for less experienced developers during implementation. For user groups with lower professional experience, the reinforcement of syntax-related issues by the suggestions generated by AI seemed to alleviate such problems and speed up the process of implementation. However, the results were also found to vary among the junior developers, especially in terms of software quality, in cases of accepting generated outputs without significant validation.

The productivity, maintainability, and software quality metrics showed the best results for intermediate developers. This group seemed to be better equipped to adopt Artificial Intelligence recommendations and to critically assess the quality of implementation and architectural consistency. Senior developers, on the other hand, gained relatively little productivity benefit but consistently had better overall software quality results. Among the experienced participants, there seemed to be a strong focus on using AI systems to complete repetitive implementation tasks, rather than for more abstract considerations of architecture or business rules.

Task-level analysis also showed that for structured implementation workflows, AI-assisted systems were significantly more effective than those involving abstract reasoning tasks. Larger gains were seen regarding repetitive enterprise-oriented programs with standardized implementation patterns, while smaller gains were seen in algorithmic reasoning, refactoring, and architecture-oriented workflows. These results are consistent with previous studies indicating that present generative AI systems excel at helping with implementation but are less capable in higher-level software-engineering reasoning [18].

While the results point to tangible benefits from using AI for development, they do not indicate that AI systems are capable of supplanting skilled software engineering practices. Some of the participants accepted generated outputs in a few instances without adequate architectural or correctness verification, signifying continued concerns over automation bias and overreliance on generative AI systems [16, 20]. Human intervention is thus still necessary to maintain and control the quality and maintainability of the software, its security, and the architectural consistency in enterprise development environments.

The results indicate that AI-based programming systems can be used successfully in enterprise-oriented .NET software engineering workflows as productivity-enhancing and implementation-support tools. These systems, however, are strongly reliant on the skills and experience of the developer(s), the nature of the task, and the capacity of the developer(s) to assess the output produced. AI systems are thus best positioned to support, rather than supplant, professional software engineering tasks and activities, as well as architectural reasoning.

VI. THREATS TO VALIDITY

Like any empirical software engineering study, there are several limitations here to be considered in interpreting and generalizing the results of the study. To enhance the transparency of research, threats to validity are discussed throughout the internal, external, construct, and conclusion validity dimensions.

A. Internal Validity

Internal validity is the question of whether the observed difference in the development conditions of traditional and AI development was actually due to the experimental treatment or to uncontrolled external factors. To minimize potential

internal validity threats during the experiment, a few modifications were made.

The first was a repeated-measures experimental design where each student was asked to do both sets of tasks under the different types of development conditions. This helped to minimize the differences between the participants and their programming skills, problem-solving methods, and prior technical experience. Further, a counterbalanced task execution sequence was used to minimize potential ordering and learning effects.

There may have been some residual learning effects on participant performance over these controls. Participants may have known the task structures or workflow more from previous experimental sessions. Likewise, participants' experience with using AI-supported programming tools may have affected their ability to use generative AIs for the experiment.

A second potential internal validity issue is the variation in the wording of the prompts and AI interaction approaches. While all participants were informed about the role of using AI-assisted tools, varying prompting techniques and validation processes may have contributed to the differences in outputs and outcomes of quality of implementation.

B. External Validity

The issues of external validity in the study involve how much the conclusions can be extended beyond the experimental environment of the study.

The experiment took a special focus on enterprise-oriented .NET software development tasks in C# and the Visual Studio environment. Thus, the results are not necessarily applicable to other programming languages, development environments, or other software engineering areas. There are inherent differences in programming effectiveness for various technology stacks, application areas, and software architectures when using AI.

Furthermore, in the context of the study, the tasks used were enterprise-oriented (programming tasks), but the experimental setting is not a perfect environment for a large-scale industrial software engineering project in which collaborative software development processes are carried out, software is maintained over a long period of time, organizational considerations are taken into account, and software is deployed for production use.

The developer sample consisted of both juniors, intermediates, and seniors, but was not large in comparison to the larger software engineering developer population worldwide. The findings might thus be generalizable to a greater or lesser extent based on variations in education, industrial experience, domain expertise, and AI-assisted tools.

C. Construct Validity

Construct validity is related to the degree to which the chosen measurement techniques reflect the desired software engineering attributes.

Productivity, software quality, maintainability, structural complexity, and cognitive workload were measured to assess them based on the quantitative metrics used by the study: task completion time, implementation errors, cyclomatic complexity, maintainability index, code smells, and NASA-TLX workload assessment. These measures are commonly used in empirical software engineering research, but they may not be able to measure every aspect of software quality and developer performance.

Software maintainability and architectural quality are multi-dimensional concepts, for instance, which can not be described by automatic software maintainability metrics and static analysis tools. Likewise, NASA-TLX measures are partially self-reported by participants and can thus be subject to subjective perception and/or individual interpretation.

Moreover, some attributes of software engineering, like the long-term maintainability of the software, its ability to scale, its security robustness, and effectiveness in software team collaboration, could not be fully assessed within the time constraints in the experiment.

D. Conclusion Validity

Conclusion validity refers to the reliability and appropriateness of the statistical analysis procedures and interpretations.

The study used paired statistical testing, repeated-measures analysis, as well as effect size estimation and reporting of confidence intervals to enhance statistical rigor and minimize the risk of wrong conclusions. The repeated-measures experimental design also enhanced statistical sensitivity by allowing for within-subject comparisons between the different development conditions.

However, the study employed a number of dependent variables, and that could add to the probability of Type 1 statistical errors even when using the conventional significance levels. Furthermore, even though the sample size and number of observations were much bigger than in many of the previous exploratory studies, future large-scale replications would give greater confidence in the long-term reproducibility of the results.

Lastly, generative AI systems are still evolving, meaning they may not perform as reliably as other types of AI. Programming tools that utilize artificial intelligence are constantly evolving, and applications of future versions of these systems may exhibit performance, capabilities, and limitations different from those used in the present study.

In general, while the study shares some drawbacks with other controlled empirical software engineering studies, it was done with a variety of methodological controls to enhance the experimental rigor, statistical reliability, and transparency of the research. The results are thus empirical insights for the actual application of AI-aided programming systems in enterprise-scale .NET software engineering applications and emphasize the need for further extensive research in this emerging field.

VII. CONCLUSION

The study was a large-scale controlled empirical assessment of AI assisting enterprise-oriented .NET software development. A repeated measures experimental design was used to have 60 developers (juniors, intermediates, and seniors) perform eight programming tasks in a traditional development environment and 960 in an AI-assisted development environment.

The results suggest that using AI to aid software development can lead to tangible gains in software

engineering productivity and at least some software quality attributes. The AI-assisted participants were typically better able to complete programming tasks, made fewer errors in their implementation, had less structural complexity, and reported less cognitive load when working with AI tools than when not using them. Moderate improvements were also observed on maintainability-related metrics, and decreases in maintainability-related code smells in several enterprise-oriented programming tasks were also observed.

Simultaneously, the study identified some key trade-offs and constraints in using AI to assist programming. While AI-generated implementations often resulted in more efficient implementations, they also had the potential to generate a slightly larger codebase and sometimes redundant abstractions or helper structures that were very wordy and needed manual polishing. Moreover, every developer's experience with AI tools and the nature of the tasks played a role in how effective these tools were. AI systems showed promise in repetitive implementation tasks or in standard enterprise processes, but had less impact on algorithmic reasoning or architecture-related tasks that involved greater levels of contextual understanding and software engineering judgment.

The study also highlighted the importance of using AI in development, not as a substitute for professional software engineering skills but as a tool for enhancing productivity and facilitating development implementation. Nevertheless, human validation was still required to ensure the outputs were correct, the architecture was consistent, the quality of the software, and to avoid excessive use of automatically generated implementations.

The results of this exploratory investigation can be compared to numerous other studies conducted in the past and differ from them in two respects: Firstly, the empirical evaluation framework used in this study is much broader, encompassing productivity, maintainability, software quality, structural complexity and cognitive workload analysis within an integrated enterprise-wide experimental environment; secondly, the study was conducted in an exploratory manner, giving rise to several new research questions. The findings are also robust and practical because of the representation of multiple developer experience groups and the larger repeated measures data set.

Nevertheless, there are some drawbacks. This experiment specifically centred on enterprise-oriented .NET development workflows and may not be generalizable to other software ecosystems, programming languages, or industrial development scenarios. Furthermore, the complexity of long-term collaborative software engineering projects with changing requirements, organizational constraints, and production deployment processes cannot be fully captured in a controlled experiment.

Future applications should thus be explored that involve the development of software using AI in larger industrial settings, in groups or teams, and in long-term software maintenance. Further research into software security, architectural quality, developer trust, and human-AI collaboration strategies would enhance the understanding of the impact of generative AI systems in the professional software engineering practice.

The findings indicate that with proper and critical use, AI-assisted programming systems can significantly contribute to supporting enterprise-oriented software engineering activities. The integration of generative AI into software development will likely continue to grow as the technology evolves, and its impact on the field will grow more significant and warrant ongoing empirical studies on the advantages, disadvantages, and future implications of software engineering with AI assistance.

VIII. References

[1] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint arXiv:2107.03374, 2021.

[2] OpenAI, “GPT-4 Technical Report,” arXiv preprint arXiv:2303.08774, 2023.

[3] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI Pair Programmers on Software Development Productivity,” arXiv preprint arXiv:2302.06590, 2023.

[4] P. Vaithilingam, T. Zhang, and E. L. Glassman, “Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models,” in Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems, New York, NY, USA: ACM, 2022, pp. 1–15.

[5] H. Pearce, B. Ahmad, B. Tan, D. Gavitt, and R. Karri, “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions,” in Proceedings of the IEEE Symposium on Security and Privacy Workshops, Los Alamitos, CA, USA: IEEE Computer Society, 2022, pp. 754–768.

[6] A. M. Dakhel et al., “GitHub Copilot AI Pair Programmer: Asset or Liability?,” Journal of Systems and Software, vol. 203, p. 111734, 2023.

[7] S. Barke, M. B. James, and N. Polikarpova, “Grounded Copilot: How Programmers Interact with Code-Generating Models,” Proceedings of the ACM on Programming Languages, vol. 7, no. OOPSLA1, pp. 85–111, 2023.

[8] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do Users Write More Insecure Code with AI Assistants?,” in Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, New York, NY, USA: ACM, 2023.

[9] T. Nguyen and A. Nadi, “An Empirical Evaluation of AI-Generated Code Quality and Maintainability,” Empirical Software Engineering, vol. 29, no. 2, pp. 1–27, 2024.

[10] S. G. Hart and L. E. Staveland, “Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research,” Advances in Psychology, vol. 52, pp. 139–183, 1988.

[11] J. D. Zamfirescu-Pereira, R. Wong, B. Hartmann, and Q. Yang, “Why Johnny Can’t Prompt: How Non-AI Experts Try (and Fail) to Design LLM Prompts,” in Proceedings of the

2023 CHI Conference on Human Factors in Computing Systems, New York, NY, USA: ACM, 2023.

[12] P. D. Ellis, The Essential Guide to Effect Sizes. Cambridge, U.K.: Cambridge University Press, 2010.

[13] T. J. McCabe, "A Complexity Measure," IEEE Transactions on Software Engineering, vol. SE-2, no. 4, pp. 308–320, 1976.

[14] B. A. Becker et al., "Programming Is Hard—or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation," Communications of the ACM, vol. 66, no. 7, pp. 70–79, 2023.

[15] T. Denny, A. Luxton-Reilly, and D. Tempero, "On the Reliability of AI-Generated Code," in Proceedings of the ACM Global Computing Education Conference, New York, NY, USA: ACM, 2023.

[16] M. Sandoval et al., "An Empirical Study of AI-Assisted Software Development," Empirical Software Engineering, vol. 29, no. 2, pp. 1–31, 2024.

[17] A. Meyer et al., "Evaluating Cognitive Load in AI-Assisted Programming Environments," International Journal of Human-Computer Studies, vol. 178, p. 103091, 2023.

[18] D. Coyle and G. Doherty, "Clinical Evaluations and Cognitive Risks of Intelligent Assistance Systems," AI & Society, vol. 38, no. 1, pp. 45–59, 2023.

[19] C. Wohlin et al., Experimentation in Software Engineering. Berlin, Germany: Springer, 2012.

[20] R. K. Yin, Case Study Research and Applications: Design and Methods, 6th ed. Thousand Oaks, CA, USA: Sage Publications, 2018.

IX. APPENDIX A — EXPERIMENTAL TASK SPECIFICATIONS

This appendix summarizes the enterprise-oriented programming tasks used during the controlled experimental evaluation. The tasks were designed to reflect realistic software engineering activities commonly encountered within professional .NET development environments.

Table 1: Experimental Programming Tasks

Task ID	Task Description	Core Requirements
T1	CRUD REST API Implementation	Create RESTful endpoints for create, read, update, and delete operations using ASP.NET Web API architecture.
T2	Authentication and Authorization	Implement JWT-based authentication and role-based authorization middleware for secured API access.
T3	Input Validation and Error Handling	Develop validation logic and centralized exception-handling mechanisms for invalid user input scenarios.
T4	Database Query Processing	Implement database interaction logic using Entity Framework and optimized query-processing workflows.
T5	File Processing and Serialization	Develop file upload, parsing, serialization, and JSON conversion functionality.
T6	External API Integration	Integrate third-party APIs using asynchronous HTTP request handling and response validation mechanisms.
T7	Algorithmic Business Logic	Implement business-rule processing involving conditional workflows and algorithmic decision-making logic.
T8	Code Refactoring and Optimization	Refactor existing implementations to improve maintainability, readability, and structural complexity characteristics.

All tasks were pilot-tested prior to the experiment to ensure comparable implementation difficulty and realistic completion feasibility within the allocated experimental session durations.

X. APPENDIX B — STATISTICAL SUMMARY TABLES

This appendix provides additional descriptive and inferential statistical summaries supporting the empirical analysis presented within the Results section.

Table 2: Descriptive Statistical Summary

Metric	Traditional Mean (SD)	AI-Assisted Mean (SD)
Completion Time	38.74 (8.91)	29.63 (10.24)
Lines of Code	117.36 (21.84)	132.47 (28.63)
Errors	3.64 (1.48)	2.11 (1.67)
Cyclomatic Complexity	5.98 (1.27)	5.01 (1.34)
Maintainability Index	72.18 (9.43)	79.41 (10.12)
Code Smells	5.34 (1.89)	3.56 (1.97)
NASA-TLX Cognitive Load	67.21 (11.36)	52.84 (13.72)

Table 3: Paired Statistical Analysis Summary

Metric	t-value	p-value	Cohen's d
Completion Time	5.42	< 0.001	1.12
Lines of Code	-4.87	< 0.001	0.81
Errors	4.94	< 0.001	0.96
Cyclomatic Complexity	3.88	< 0.001	0.74
Maintainability Index	-4.65	< 0.001	0.88
Code Smells	4.12	< 0.001	0.79
Cognitive Load	5.91	< 0.001	1.18

Effect sizes were interpreted according to conventional thresholds for small (0.2), medium (0.5), and large (0.8) effects.

Table 4: Repeated-Measures ANOVA Summary

Metric	F-value	p-value
Completion Time	29.38	< 0.001
Lines of Code	23.16	< 0.001
Errors	25.74	< 0.001
Cyclomatic Complexity	15.05	< 0.001
Maintainability Index	21.82	< 0.001
Code Smells	17.41	< 0.001
Cognitive Load	34.57	< 0.001

The repeated-measures analysis demonstrated statistically significant differences across most evaluated productivity, software quality, maintainability, and cognitive workload metrics under AI-assisted development conditions.